CROWN FIRE ·ACTIVE CRISIS

When the Canopy Burns

A Playbook for Engineering Organizations in Active Crisis

> *You can't hire your way out of a crown fire. You need a different kind of intervention.*

UNDERSTORY COLLABORATIVE · WHAT'S ON FIRE? FIELD GUIDE

If you're reading this, the fire is visible. Leadership knows. Customers know. Your team knows. The question is no longer whether something needs to change. The question is what to change first, and how to do it without making things worse.

This paper is for the leader who's in it right now. Not the one who suspects there might be a problem. The one who's managing escalations, watching releases slip, and fielding questions from the board about why things aren't getting better despite the investment. The one whose team is spending more energy keeping things running than building what's next.

> **Crown fires are addressable. Organizations come back from this. The path back requires discipline, a rigorous assessment, and a willingness to resist the instinct to move fast just because everything feels urgent.**

How Crown Fires Develop

Crown fires don't start in the canopy. They start on the ground. A smolder that nobody addressed. Deployment friction that compounded. Knowledge concentration that went unmanaged. One-customer features that accumulated. Tech debt that the team raised and leadership deferred.

At some point, the conditions shifted. A key person left. A big customer arrived. The market demanded speed the platform couldn't deliver, and the brush that had been accumulating quietly caught fire in a way that was no longer contained to the understory.

What makes a crown fire different from a smolder is visibility. In a smolder, the team knows and leadership doesn't. In a crown fire, everyone knows. Leadership is involved in technical decisions they shouldn't need to be involved in. Releases are events that require executive attention. Customer escalations are routine, and when a significant opportunity shows up, the reaction across the organization is anxiety, because nobody is confident the platform can support it.

> **The dangerous thing about a crown fire is the decisions people make in response to it.**

Why Adding Headcount Usually Makes It Worse

The most common response to a crown fire is to hire. The logic feels sound. We're short on capacity, we need more people, more people means more output. In a healthy organization, that logic holds. In a crown fire, it inverts.

Picture the actual situation on the ground. Your projects are already behind schedule. Your developers are just trying to stay above water, keeping escalations at bay while slowly inching forward on feature work. Your QA team is barely keeping up with releases and work in development simultaneously. Dev leads, managers, and product owners are spending their energy trying to protect the teams from getting further off track.

Now you add Dave. Dave knows nothing about the processes, the products, the projects, or the team. He's talented. He's eager. You have two options for what happens next.

Option one: invest in Dave. The team takes time to have Dave shadow team members and learn the ropes, but every fifth process, product detail, or policy has to be explained, and that's being charitable. The team is now back in forming and possibly storming mode. They're on guard because someone new has joined and they're spending brain cycles trying to figure out how this person is going to fit with the existing dynamic. The whole team slows by 20% in order to get Dave up to speed. By the time he is, they've blown past the target date.

Option two: ignore Dave. The team keeps going at their regular pace and lets Dave figure it out on his own. They deliver barely on time. They have to squash a mountain of bugs once the feature is released. Three months later, Dave hates working here. There's a rift in the team that might never heal, because this was his first impression of the organization, and by the time the team catches their breath they feel like Dave has been around long enough that he should know more than he does, and here comes the next fire.

There is no option three. In a crown fire, those are the choices. The team absorbs the cost of onboarding and falls further behind, or the team ignores the new person and creates a new organizational problem on top of the existing technical ones.

There is the contractor variation, which might be worse. Leadership brings in an entire contract team to work in legacy code and promises that the core teams will only have to consult. Maybe review some PRs. In practice, those "consultations" become a significant and untracked time sink. The core team is surprised by poor coding technique. They're watching the contractors replicate patterns the team has been actively trying to move away from. Nobody is measuring how much time the core team is actually spending on guidance and review, so everyone is surprised when they report they'll be a month and a half behind on delivering a key feature for busy season.

> **The point is timing. Adding people as a crisis response is what backfires. Hire when the foundation is stable enough for new people to stand on. Bring in contractors when the core team has enough bandwidth to actually guide them.**

Assessment Before Action

When things are actively on fire, the instinct is to start fixing immediately. Pick the thing that's burning brightest and throw resources at it. Move fast. Show progress.

Resist this.

We've walked into organizations where the stated problem was "we need to fix our deployment pipeline and migrate to AWS." Sometimes the deployment pipeline does need fixing and the migration does need to happen, but when we sit down and actually map what's happening, we find that the deployment pipeline is not the root cause of the crisis. The root cause is that one person in the middle of a multi-department, business-critical process is performing ten manual steps that nobody else knows how to do. No amount of pipeline improvement changes that. You can build the most elegant CI/CD system in the world and it will not matter if the entire workflow stops when one person goes on vacation.

The pushback we hear most often is "we don't have time for an assessment. We're drowning." The answer is: you were drowning before you hired us. A couple of weeks to make sure you're fixing the right things is going to save more in the long run than rushing to a solution that lacks context. Applying the same fix to every organization is what amateurs do. The problems

may look similar from the outside, but the root causes are specific to your people, your systems, and your history.

✓

What an assessment looks like:

Map the actual workflow, not the documented one. Get every person involved in a critical process into the same room and walk through it step by step. You will find steps that nobody knew existed. You will find single points of failure. You will find tasks that were assigned to people who are no longer there. The difference between the process you think you have and the process you actually have is where the real problems live.

Identify what's on fire versus what's smoldering versus what's already burned. Not everything that feels urgent is equally important. Some systems are actively failing and need immediate attention. Some are degraded but stable enough to wait. Some have already failed and the organization has routed around them. Treating everything as equally urgent guarantees you'll fix none of it well.

Find the load-bearing people. In every crown fire, there are a small number of people holding a disproportionate amount of the system together. They're the ones getting paged at 2 AM. They're the ones who get pulled into every incident. They're the ones who can't take vacation because nobody else can do what they do. These people are simultaneously your most important asset and your biggest risk. If you lose them, the crown fire becomes a firestorm.

Separate symptoms from causes. Slow releases are a symptom. Frequent escalations are a symptom. The causes are further upstream. Missing automation, knowledge concentration, architectural decisions that made sense five years ago and don't anymore, organizational structures that create bottlenecks. Fix symptoms and they come back. Fix causes and the symptoms resolve.

The Triage Framework

Once you've assessed, you need a framework for deciding what to address first. In a crown fire, you cannot fix everything at once. You have to choose, and the choices have to be deliberate.

Stabilize first, improve second, build third.

The instinct is to jump to improvement or building because those feel like progress. Stabilization feels like treading water. You cannot improve a system that's actively failing, and you cannot build on a foundation that's shifting. Stabilization means getting to a state where the current fires are contained and new fires aren't starting faster than you can respond to them.

Prioritize by dependency, not by pain.

The most painful problem is not always the most important one to fix first. Look for the problems that block other fixes. If your deployment process is so unreliable that you can't safely ship changes, then every other fix is blocked by that, because you can't deliver the fixes reliably. If knowledge concentration means only one person can troubleshoot production issues, then every incident response is bottlenecked by that person's availability. Find the constraint that constrains everything else, and start there.

Decide what to let burn.

This is the hardest part. In a crown fire, some things are going to continue to be broken for a while. You need to consciously decide which problems you're not going to address right now, communicate that decision to the team and to stakeholders, and accept the consequences. The alternative is spreading your limited capacity across too many fronts and making meaningful progress on none of them.

Decide what to let go entirely.

Some systems, features, or commitments are not worth saving. The custom implementation from five years ago that one customer uses and nobody understands? It may be time to sunset it. The internal tool that three people use but requires constant maintenance? It may be time to replace it with something off the shelf or retire it altogether. Crown fires are

clarifying moments. They force you to ask which things are actually essential and which things you've been carrying out of inertia.

Sometimes what you have to let go of is new features. This is where the conversation gets uncomfortable, because new features are what leadership gets excited about. They're what sales has promised. They're on the roadmap. If you have a compliance problem that could cost millions in fines and lawsuits, compliance has to win. It is not sexy work. Nobody is going to celebrate it in the next all-hands. The new feature does not matter if the business is buried in regulatory penalties or legal action. You have to be willing to give up the things that feel like progress in order to keep the business afloat. That decision has to come from leadership, explicitly, so the team is not left trying to do both and failing at both.

Managing Stakeholders During Crisis

Technical crises create communication challenges at every level of the organization. Leadership wants to know when it will be fixed. Customers want to know why things are broken. The team wants to know that someone has a plan.

With leadership and the board:

Be upfront about the timeline and the tradeoffs. "We can stabilize the most critical systems in 30 days, but some lower-priority issues will persist for 90 days or more." This is a harder message to deliver than "we'll have it fixed soon," but it builds trust and it sets expectations that you can actually meet. Overpromising in a crisis is how you turn a technical problem into a credibility problem.

With customers:

Acknowledge the impact without over-explaining the technical details. Customers don't need to understand your deployment pipeline. They need to know that you understand their experience is degraded, that you're working on it, and that you'll communicate progress. Regular, brief updates are better than infrequent detailed ones.

With the team:

The people closest to the fire need two things. A clear priority order so they know what to work on, and visible leadership engagement so they know the organization takes this

seriously. If leadership is asking the team to fix the crisis while simultaneously pushing for new features on the same timeline, the message is that the crisis isn't actually a priority. Protect the team's focus. Make the hard calls about what gets deferred so they don't have to.

Communicate in both directions.

This is one of the most common leadership failures in a crisis, and it often comes from good intentions. A leader fights hard behind closed doors to get their team more time, more resources, or more protection from competing priorities. They win some of those battles. They never tell the team what they did, because they don't want to seem like they're asking for credit, or because they assume the team can see the results.

The team cannot see the results. What the team sees is that they raised a problem and nothing visibly changed. They feel unheard. They lose trust. The leader who was genuinely advocating for them has no idea that the team doesn't know.

Actions are always louder than words, but actions that happen in rooms your team is not in are invisible. You have to communicate what you're doing, what you've secured, and what you're still fighting for. If you've already burned your credibility through too many "we'll get to it" promises that never materialized, words alone will not rebuild it. You have to show something concrete. Cancel a feature to make room for stabilization work and tell the team why. Move a deadline and explain what that cost you politically. Put real action on the line where the team can see it. That's how you rebuild trust during a crisis.

What a Crown Fire Does to the Team

In a smolder, people stop trusting that problems will be addressed. In a crown fire, people stop trusting each other.

This is a predictable response to sustained crisis. Nobody is failing at their job. When the team is in survival mode, the collaborative behaviors that make a healthy team function start to break down. People stop helping each other with problems outside their immediate scope, not because they don't care, but because they don't have the capacity. Everyone is fully consumed keeping their own area from collapsing. The instinct to reach across and help a teammate is replaced by the instinct to protect your own deliverables, because those are the ones you'll be held accountable for.

The team stops solving problems together. In a healthy environment, a hard problem gets brought to the group. People contribute ideas. Someone takes the lead. The solution draws on the collective experience of the team. In a crown fire, hard problems get handled by whoever is closest, alone, under pressure, with whatever knowledge they have. There's no time to collaborate. There's barely time to communicate. People make decisions in isolation and hope they made the right call.

This isolation compounds the knowledge concentration problem. In a smolder, knowledge silos form gradually through neglect. In a crown fire, they form actively. People become experts in their crisis, their system, their fires. The barriers between those areas harden because nobody has the bandwidth to cross them. A team that used to function as a unit fragments into individuals managing their own emergencies.

Then there's the effect on new people. The Dave scenario from the headcount section plays out against this backdrop. You're asking a new person to integrate into a team that has stopped functioning as a team. The existing members don't have the energy to build relationships with someone new. They don't have the context to spare. The new person reads the room and either forces their way in, creating friction, or retreats, becoming isolated. Neither outcome produces a stronger team.

The most corrosive effect of a crown fire on the team is what happens to their belief that problems are solvable at all. In a brush fire, the team believes things can improve but nobody's prioritizing it. In a smolder, the team has stopped believing leadership will prioritize it. In a crown fire, the team is starting to wonder whether the problems are even fixable. They've been fighting fires for so long that the idea of working on something other than the current emergency feels abstract. The muscle for proactive improvement has atrophied. Even if you gave them protected time tomorrow, it would take weeks before the team could shift out of reactive mode and start using it productively.

Recovering from this requires more than technical stabilization. It requires giving the team evidence, real evidence, that the crisis is being taken seriously, that the workload is going to change, and that collaboration is possible again. You have to create the conditions for the team to start functioning as a team again, because the crisis has systematically dismantled those conditions.

The 90-Day Stabilization Pattern

Ninety days is not arbitrary. It's a realistic timeline for getting from active crisis to a state where the organization is reactive but no longer in freefall. It will not fix everything. It will get you to a place where you can start making intentional decisions instead of emergency ones.

Days 1 through 14: Assess and triage.

Map the real state of things. Identify the critical failure points. Talk to the people closest to the systems. Build the priority list. Resist the urge to start fixing before you understand what's actually broken.

This is the phase where you'll face the most pressure to skip ahead. "We already know what's wrong, just start fixing it." The assessment almost always reveals things that contradict what leadership believes the problems are. Those two weeks buy you the clarity to fix the right things in the right order.

Days 15 through 45: Stabilize the critical path.

Address the top two or three issues from your triage. These are the problems that either affect customers directly or block every other improvement. The goal is not perfection. The goal is to stop the active bleeding and establish a baseline where new incidents are not outpacing your ability to respond.

This is also when you start addressing the load-bearing people problem. Cross-train where you can. Document the critical knowledge. Begin shifting from a model where one person holds everything to a model where at least two people can handle any critical function.

Days 46 through 90: Build the foundation for sustained improvement.

With the critical fires contained, you can start working on the underlying causes. This is where you address the deployment pipeline, the test coverage, the monitoring gaps, the architectural debt. Not all of it, but the pieces that will have the highest leverage for ongoing stability.

By day 90, the goal is a team that can describe their current state plainly, has a prioritized plan for continued improvement, and is no longer operating in crisis mode. There will still be significant work to do. The difference between "we have a plan and we're executing it" and

"we're drowning and we don't know where to start" is the difference between a recoverable situation and one that drives your best people out the door.

When to Bring in Outside Help

There's no shame in bringing in specialists. You wouldn't expect your primary care doctor to perform surgery. The question is how to evaluate outside help without getting burned again.

✓

Look for people who insist on assessing before prescribing. Anyone who tells you what to fix before they've looked at your specific situation is selling you a solution they already have, not the one you actually need.

Look for people who've seen your pattern before. Crown fires share common characteristics, but the details matter. Experience with organizations at your stage, your scale, and in your general domain means less time spent learning your context and more time spent on the actual problems.

Look for people who plan to leave. The right outside help works themselves out of a job. They stabilize, they transfer knowledge, they build your team's capacity to continue without them. If someone's engagement model depends on you needing them indefinitely, their incentives are misaligned with your recovery.

Be wary of the "rewrite everything" recommendation. In a crown fire, the temptation to start over is enormous. Sometimes a rewrite or a major re-architecture is genuinely the right call. More often, it's a way to avoid the harder work of understanding and improving what exists. A full rewrite is a multi-year commitment with its own risks. Make sure you've exhausted the alternatives before committing to one.

Now what

You have a read on how serious this is. The next step is turning it into a plan for the one thing most in your way. As you read, mark this up: what resonated, what didn't fit, and what you're still unsure about. Then write down the three questions you still have.

Bring your question to TPM office hours →

Not ready for that? Ask us a question for free. We answer the ones that help the most people, on the blog and on socials.

About Understory Collaborative

Understory Collaborative is a team of seasoned technologists who specialize in the work that happens beneath the surface. We're smokejumpers. We help organizations find the root cause of a crisis, stabilize what's critical, and build the foundation for sustained recovery.

We assess before we prescribe. We've seen these patterns before, and we plan to leave your organization stronger than we found it.

If anything in this paper sounded familiar, we should talk. Reach out at contact@understorycollab.com or visit understorycollab.com/contact.